

SERVICE ACTUEL · PROCESS-DATA

Exécution des flows, planification et ressources de calcul

22 capacités analysées

services/process-data · Kotlin 2.4.10, Spring Boot 4.1.1 et JVM 21 observés dans Gradle. PostgreSQL/JPA/Flyway, Kafka/Protobuf avec outbox/inbox, Quartz JDBC clusterisé pour cron, boucles Spring de réconciliation, S3, REST app-management et HTTP vers les processus de session Python/PySpark.

Service qui reçoit les versions exécutables produites par data-dev, crée et suit les runs et jobs, orchestre les sessions Python/PySpark, gère les déclenchements, clusters et images, expose historique et temps réel, et relaie les états aux autres services.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Runtimes, Flows, IAM, Writer, Publisher, Query, Applications, Notifications, Audit, Migration, Maintenance, Egress.

Périmètre de lecture

- Controllers execution, schedule, session, image et admin, DTO de soumission et filtres.
- Agrégats DataflowVersion, Script, ExecutionStorageManifest, DataflowExecution, jobs, files persistantes, Cluster, Session et schedules.
- Handlers de run manuel/automatique/partiel, soumission, suivi, logs, fin de job, annulation et callback Spark.
- Adaptateur app-management et appels directs de sessions, cycle de clusters/images, bibliothèque PyPI, startup et réconciliation.
- Consumers DataDev/ProcessData/IAM, outbox/inbox/DLT, maintenance PostgreSQL, RunnerAuthorizationClient, manifestes Gradle et chart.

Objets et cycle de vie

- DataflowVersion : copie exécutable reçue de data-dev, avec scripts, dépendances, manifestes, propriétaire, branche et projet réel d'exécution ; ce n'est pas une édition de données publiée.
- DataflowExecution : run avec statut, déclencheur, principal, cluster, échéance de capacité, dates et jobs. L'exécution partielle conserve des résultats antérieurs marqués comme réutilisés.
- DataflowJobExecution et JobDetails : script, étapes, sorties, logs et flag historique isSparkCommitted. Les statuts finaux ne décrivent pas à eux seuls un reçu de publication.
- DataflowSubmission, JobSubmission, JobSubmitted, JobCompleted et ExecutionCancelled : intentions et étapes de progression durables utilisées pour reprendre après incident.
- Cluster et Session : capacité persistante Python/PySpark, ressources, images, état observé, pause et suppression différée.
- Image, ImageUpdate et Library : base, environnement personnalisé, demandes de packages, builds et historique.
- DataflowSchedule : expression/fuseau ou déclenchement événementiel ; Quartz stocke jobs, triggers et heartbeat.
- Miroirs User/Project/ProjectMember, correspondances IAM, outbox, inbox, quarantaine, chunks de logs et suivi de migration.

Dépendances du service

- Data-dev produit VersionEvent, changements de propriétaire, suppressions et triggers ; process-data renvoie états de run, schedules et clusters.
- App-management reçoit les appels REST de session, serveur Spark Connect, submission, cancellation, inventaire et build d'image ; les sessions exposent aussi leur état/job en HTTP.
- S3 héberge les archives de scripts ; le run obtient une URL signée vers la version/script. Le contenu effectif et les versions déployées ne sont pas inspectés.
- IAM autorise les opérations de runner et fournit les miroirs ; gateway protège les routes externes ; Notification Center relaie les mises à jour de clusters.
- PostgreSQL conserve contrôle et historique ; Kafka transporte Protobuf et événements ; Quartz ajoute la planification clusterisée ; PyPI/index interne fournit les bibliothèques.

Fonctionnalités et compatibilité à préserver

PD-01 **Cible non qualifiée**

Réception des versions et intégrité du manifeste

COMPORTEMENT OBSERVÉ

Le consommateur VersionEvent réclame une clé d'inbox dans sa transaction puis construit une version locale. VERSION_SAVE conserve la version sans run. MERGE, PUBLISH, COLLECT, PREVIEW, DELETE, EMPTY_DATASET et UPDATE_COLUMN peuvent lancer automatiquement le traitement si le démarrage des clusters est actif. Les actions inconnues sont rejetées pour retry/DLT. Une version dupliquée ne ressuscite pas une version supprimée. Le handler peut enrichir uniquement un manifeste absent ; il vérifie le même ensemble de scripts et refuse un manifeste non nul divergent. Il préserve les autres métadonnées de la version existante. La portée execution_project_id vient du producteur et une valeur invalide ne devient pas un runner global.

UTILISATEURS ET RÔLES

Data-dev, service de migration et exécuter.

INTERFACES DANS LE CODE

Kafka VersionEvent ; DataDevKafkaEventConsumer.consumeVersionEvent ; DataflowVersionCreateCommandHandler ; VersionEventProtoConverter.

DONNÉES ET ÉTATS

DataflowVersion immatriculée par VersionId, branches et projet, scripts avec dépendances et manifestes ; inbox de déduplication ; marqueur deletedAt.

DÉPENDANCES ET ÉCHANGES

Data-dev par Protobuf/Kafka, PostgreSQL et contrôleur de runs.

COMPORTEMENT CIBLE PROPOSÉ

Flows/Runtimes Rust acceptent un RunSpec immuable et versionné. Un même ID ne peut désigner des artefacts ou droits différents. Les versions anciennes sans manifeste sont isolées jusqu'à preuve ou conversion explicite.

MIGRATION DE CETTE CAPACITÉ

Importer versions, scripts, manifestes et inbox de manière cohérente. Conserver les références d'origine mais remplacer les IDs synthétiques d'opérations techniques par un type de run explicite. Ne jamais reconstruire des droits depuis le flow courant.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Rejouer la même version avec un manifeste égal, absent puis divergent. Les deux premiers cas conservent les droits existants ; le troisième est refusé. Rejouer une version supprimée ne la remet ni en file ni en service.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/event/consumer/DataDevKafkaEventConsumer.kt:45](#) Inbox, stockage et déclenchement conditionnel.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowVersionCreateCommandHandler.kt:44](#)

Immutabilité et enrichissement limité du manifeste.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/event/consumer/converter/VersionEventProtoConverter.kt:142](#) Portée projet et matrice d'actions automatiques.

PD-02 **Cible non qualifiée**

Lancement manuel d'un flow complet**COMPORTEMENT OBSERVÉ**

Le lancement manuel attend que la version demandée soit visible, refuse une version supprimée et une version sans script, puis consulte la dernière exécution de sa branche. Si elle est CREATED, PENDING ou RUNNING, il refuse un nouveau run concurrent selon la règle métier locale. Le cluster vient de l'override de requête, de la version ou du précédent run, avec repli vers un cluster par défaut du type attendu. Le service crée un DataflowExecution, calcule son nom numérique par comptage des exécutions du projet, persiste ses jobs éventuels puis émet ExecutionCreatedEvent. Le demandeur authentifié devient le principal MANUAL_CALLER distinct du propriétaire de version.

UTILISATEURS ET RÔLES

Utilisateur autorisé à exécuter le flow.

INTERFACES DANS LE CODE

POST /dataflows/{id}/versions/{version}/run, paramètre cluster optionnel ;
DataflowVersionRunManualCommand.

DONNÉES ET ÉTATS

Execution CREATED puis événement vers le traitement de file ; principal capturé, cluster, version, timestamps et deadline.

DÉPENDANCES ET ÉCHANGES

Versions répliquées de data-dev, clusters, miroirs utilisateurs/projets, outbox d'exécution et notifications.

COMPORTEMENT CIBLE PROPOSÉ

Runtimes Rust accepte une commande idempotente par projet et version, acquiert une réservation atomique de concurrence et fige RunSpec. Le run est borné en temps, octets et ressources, indépendamment du pod worker.

MIGRATION DE CETTE CAPACITÉ

Conserver la politique de concurrence par branche ou la faire évoluer explicitement. Remplacer le contrôle lecture-puis-écriture par une contrainte/lease transactionnelle. Corréler le flow de l'URI avec la version persistée dans le nouveau contrat.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Envoyer deux requêtes concurrentes de même clé de commande sur deux replicas. Elles retournent un seul run. Deux clés différentes visant une branche limitée à un run produisent un gagnant et un conflit déterministe.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/execution/controller/DataflowPipelineController.kt:206](#) Route et identité du demandeur.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowVersionRunCommandHandler.kt:198](#)

Conditions de lancement et choix du cluster.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowVersionRunCommandHandler.kt:101](#) Création, persistance et événements du run.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/session/aggregate/Cluster.kt:196](#) Sélection d'un cluster ou d'un défaut par type.

PD-03 Cible non qualifiée**Exécution partielle d'une transformation et réutilisation d'amont****COMPOTEMENT OBSERVÉ**

La route de transformation résout le script correspondant à l'identifiant métier de transformation. Elle refuse un dernier run de branche encore actif, choisit un cluster puis calcule les dépendances à relancer. Un amont dont le job précédent est SUCCESS ou WARNING n'est pas ajouté à la fermeture de dépendances. Le nouveau run peut copier des jobs du précédent avec de nouveaux IDs et `isFromAnotherExecution=true`, tout en relançant les scripts sélectionnés. Les dates de démarrage, la session et la deadline sont renouvelées. Le résultat est marqué `isPartialRun` ; le statut global ignore certains jobs copiés dans son calcul d'échec.

UTILISATEURS ET RÔLES

Développeur qui corrige ou inspecte une transformation.

INTERFACES DANS LE CODE

POST `/dataflows/{id}/versions/{version}/transformations/{transformation}/run` ; `DataflowVersion.getDepends` ; `DataflowExecution.singleScriptExecution`.

DONNÉES ET ÉTATS

Nouveau run partiel, jobs réutilisés marqués, nouveaux jobs pour le sous-graphe nécessaire et nouvelle attribution du lancement.

DÉPENDANCES ET ÉCHANGES

Historique des jobs de branche, graphe de dépendances de la version et runtime de session.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes Rust réutilise uniquement des résultats liés à des éditions d'entrée et artefacts immuables prouvés. La sélection d'un sous-graphe enregistre les dépendances réutilisées, leur provenance et la politique d'invalidation.

MIGRATION DE CETTE CAPACITÉ

Ne pas convertir `isFromAnotherExecution` en preuve suffisante de réutilisation. Importer cette information comme provenance historique, puis exiger digests, éditions et paramètres pour les nouveaux runs partiels.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Relancer une transformation après modification d'une entrée amont. Le runtime invalide l'ancien résultat et recalcule le sous-graphe requis. Si aucun élément du manifeste n'a changé, il réutilise l'édition exacte et l'explicite dans le reçu du run.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowVersionRunCommandHandler.kt:163](#) Sous-graphe, dernier run et principal du run partiel.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowVersion.kt:68](#) Amonts SUCCESS/WARNING réutilisés.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowExecution.kt:83](#) Copie de jobs et renouvellement du run.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowExecution.kt:137](#) Calcul de résultat partiel.

PD-04 **Cible non qualifiée**

Identité d'exécution, projet réel et durée d'autorisation**COMPOTEMENT OBSERVÉ**

L'exécution conserve ExecutionPrincipal avec MANUAL_CALLER, SCHEDULED_OWNER ou INTERNAL_OWNER. Les runs planifiés capturent le propriétaire de la version utilisée ; les manuels capturent le demandeur. La deadline d'autorisation est fixée à la création et le repository conserve celle de la ligne verrouillée lors des sauvegardes. Submit refuse de lancer un job après cette borne. Pour les jobs de flow/preview, le projet vient de la version ; pour collecte/suppression/vidage/colonnes, il vient de executionProjectId, distinct des anciennes références synthétiques. Le service n'envoie pas directement le manifeste dans SubmitJobRequestDTO ; app-management reçoit les IDs du job et doit résoudre les droits selon son propre contrat.

UTILISATEURS ET RÔLES

Utilisateur, scheduler, run technique, IAM et app-management.

INTERFACES DANS LE CODE

ExecutionPrincipal ; DataflowExecution.capabilityDeadline ; SubmitCommandHandler ; SubmitJobRequestDTO.

DONNÉES ET ÉTATS

Principal et deadline persistés dans dataflow_execution ; version et type de job ; projet d'exécution réel optionnel pour l'historique.

DÉPENDANCES ET ÉCHANGES

IAM et broker de credentials via app-management ; miroirs de projet ; manifeste de scripts conservé en base.

COMPOTEMENT CIBLE PROPOSÉ

IAM/Runtimes Rust délivrent une capacité liée à tenant, projet, run, tentative, manifestes et échéance. La révocation coupe les nouveaux accès. Les workers sont fongibles et ne possèdent ni credentials de catalogue génériques ni état autoritaire.

MIGRATION DE CETTE CAPACITÉ

Importer le principal seulement quand il est prouvé. Les exécutions anciennes sans principal ne doivent pas recevoir de fallback global. Fixer le protocole de résolution app-management et la borne de renouvellement avant remplacement.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Changer le propriétaire après lancement d'un run puis réessayer une soumission. L'identité d'origine reste inchangée. Après l'échéance, aucune nouvelle délégation n'est émise et la tentative se termine avec un motif contrôlable.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowExecution.kt:20](#) Types de principal et deadline.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/execution/repository/DataflowExecutionRepositoryImpl.kt:76](#) Préservation de la deadline sur ligne verrouillée.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SubmitCommandHandler.kt:99](#) Borne de soumission et choix du projet réel.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/session/service/appmanagement/dto/RequestDTO.kt:67](#) Contrat de soumission sans valeur de credential.

PD-05 Cible non qualifiée**Files persistantes et progression du graphe de jobs****COMPOTEMENT OBSERVÉ**

ExecutionCreatedEvent fait passer une exécution CREATED à PENDING et crée DataflowSubmission. La file est ordonnée par cluster et expose une position. Le démarreur charge les clusters et jobs déjà soumis, cherche une session libre et crée des JobSubmission pour les racines du graphe. Une session est choisie parmi les IDLE sans job en attente/soumis ; les jobs d'un même flow peuvent rester rattachés à sa session. Les enfants sont déclenchés après succès et seulement si tous leurs amonts sont SUCCESS. Le traitement parcourt successivement nettoyage, sessions, annulation, démarrage, soumission, suivi et fin de job. Cette boucle est un @Scheduled du pod ; elle n'est pas le job Quartz utilisé pour les cron.

UTILISATEURS ET RÔLES

Ordonnanceur interne et opérateur suivant les files.

INTERFACES DANS LE CODE

DataflowExecutionRunCommandHandler ; StartExecutionCommandHandler ;
Orchestrator.manageOrchestratorLifecycle.

DONNÉES ET ÉTATS

DataflowSubmission, JobSubmission, JobSubmitted, JobCompleted et ExecutionCancelled en PostgreSQL ;
run CREATED/PENDING/RUNNING.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, ClusterService et événements de succès/session.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes Rust possède un ordonnanceur durable avec claims atomiques, leases bornés, tentative monotone et déduplication. Les files ne donnent pas de droits au stockage. Le worker remplaçable ne modifie jamais le checkpoint publié.

MIGRATION DE CETTE CAPACITÉ

Conserver les identifiants de commande/job lors de la transition. Remplacer les séquences d'appels au repository par des transactions définies. Qualifier la fenêtre entre création distante du job et enregistrement local avant montée en replicas.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Perdre un pod entre réservation d'une session et création distante du job. Un autre contrôleur reprend après expiration de lease ; un seul effet de soumission est accepté et la file ne reste pas bloquée.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowExecutionRunCommandHandler.kt:82](#) Passage PENDING et file persistante.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/StartExecutionCommandHandler.kt:84](#) Déqueue, session et racines.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/session/aggregate/Cluster.kt:156](#) Sélection de session libre.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/execution/service/Orchestrator.kt:45](#) Boucle périodique du pod.

PD-06 **Cible non qualifiée****Soumission aux sessions Python et Spark Connect****COMPOTEMENT OBSERVÉ**

Submit examine JobSubmission et évite un nouveau job si la session possède déjà un JobSubmitted. Il exige une session IDLE, charge l'exécution, refuse un run final ou hors deadline et soumet via ClusterService. L'adaptateur crée une URL S3 présignée pour l'artefact version/script puis envoie job, cluster, session et projet à app-management. Une création distante valide devient job RUNNING, JobSubmitted et session BUSY. Une session absente, invalide ou une réponse nulle déclenche FAILURE. Le statut et l'annulation du job passent ensuite directement par HTTP vers le processus de session. Les appels distants et écritures PostgreSQL ne forment pas une transaction atomique dans les handlers lus.

UTILISATEURS ET RÔLES

Ordonnanceur, app-management et processus de calcul.

INTERFACES DANS LE CODE

POST app-management /private/sessions/jobs/submit ; HTTP de session submit/{statementId} ; ClusterConnectServerServiceAppManagementImpl.submitJob/getJobState/cancel.

DONNÉES ET ÉTATS

JobSubmission vers JobSubmitted avec StatementId distant ; session IDLE/BUSY ; artefact S3 référencé par URL courte.

DÉPENDANCES ET ÉCHANGES

App-management authentifié par client de service, S3, réseau de sessions Python/PySpark.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes Rust soumet un JobSpec à un adaptateur Kubernetes/processus non JVM. L'ID de tentative est idempotent côté exécuter ; la réponse distante peut être retrouvée après perte réseau. L'artefact est vérifié par digest avant exécution.

MIGRATION DE CETTE CAPACITÉ

Préserver les contrats de job et les reçus pendant le remplacement de l'adaptateur. Introduire une opération query-by-attempt pour résoudre une soumission ambiguë. Remplacer les sessions Spark Connect par workers bornés et pools Python adaptés.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Faire créer un job distant puis couper la réponse. La reprise retrouve le même job par son ID de tentative, vérifie son digest et ne le crée pas une deuxième fois. Une URL d'artefact expirée donne une erreur reprise sans fallback de credentials.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SubmitCommandHandler.kt:56](#) Garde de session et transitions locales.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SubmitCommandHandler.kt:128](#) Traitement des résultats de soumission.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/session/service/appmanagement/ClusterConnectServerServiceAppManagementImpl.kt:173](#) Présignature et soumission distante.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/session/service/appmanagement/ClusterConnectServerServiceAppManagementImpl.kt:198](#) Suivi et annulation HTTP directs.

PD-07 **Cible non qualifiée**

États des jobs, étapes et sorties**COMPOTEMENT OBSERVÉ**

Le run et les jobs utilisent CREATED, PENDING, RUNNING, FAILURE, SUCCESS, SKIPPED, WARNING et CANCELED. Les états finaux d'un job ne sont pas remplacés par un nouvel état. Les détails associent transformation, étape, sortie, logs et un outputStatus distinct. Le suivi de session convertit available en succès, error en échec et un état inconnu en échec. Un job échoué marque les descendants non réussis SKIPPED. Une réussite déclenche les enfants admissibles. Le run devient terminal quand tous les scripts possèdent un job final ; WARNING peut contribuer à l'échec global, avec une règle propre aux runs partiels. Les messages Python renseignent aussi les étapes sans déterminer seuls le résultat global.

UTILISATEURS ET RÔLES

Exécuteur, bibliothèque Python, frontends et consommateurs de datasets.

INTERFACES DANS LE CODE

ExecutionUpdateStatusCommandHandler ; JobCompletedCommandHandler ; ScriptExecutionEvent ; ExecuteFinalPipelineEvent.

DONNÉES ET ÉTATS

DataflowExecution, DataflowJobExecution, JobDetail et ExecutionState de sortie. Historiques terminal et de provenance des jobs repris.

DÉPENDANCES ET ÉCHANGES

Runtime de session pour résultat, Kafka pour messages de scripts et événements de fin, projection data-dev/data-model.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes Rust sépare état du calcul, état du candidat, décision d'audit et publication. Une sortie n'est disponible au lecteur que par une édition approuvée. Le vocabulaire legacy peut être exposé par une projection de compatibilité.

MIGRATION DE CETTE CAPACITÉ

Écrire une table de transition et une matrice de conversion des états inconnus, WARNING, SKIPPED et reprises partielles. Éviter de recopier SUCCESS dans un statut de publication. Corréler les événements au run/tentative, pas seulement à version/script.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Livrer successivement RUNNING, SUCCESS puis un RUNNING retardé pour une même tentative. Le job reste terminal. Un run avec un enfant sauté et un candidat refusé expose séparément ces faits et ne publie aucune sortie partielle non approuvée.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowExecution.kt:137](#) Calcul de fin et état global.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowExecution.kt:488](#) Monotonie des statuts de jobs.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/ExecutionUpdateStatusCommandHandler.kt:187](#) Succès, échec, descendants et messages d'étape.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/JobCompletedCommandHandler.kt:50](#) Interprétation des résultats de session.

PD-08 **Cible non qualifiée****Logs bornés, historique des transformations et conservation finale****COMPOTEMENT OBSERVÉ**

Les messages de scripts écrivent des chunks associés à run/job/étape dans `dataflow_job_execution_log`. La normalisation gère les retours de ligne réels et échappés ; le traitement borne les lignes et peut filtrer les lignes marquées administrateur. Le poll de session extrait stdout, message et traceback. À la fin du job, le handler reconstruit les logs finaux, les sauvegarde dans les détails puis peut retirer les chunks déjà compactés. Les routes permettent détail du run, logs d'une transformation et choix `showAll/showAllLogs`. Des logs finaux tardifs peuvent encore être ajoutés alors que des transitions non finales tardives sont ignorées.

UTILISATEURS ET RÔLES

Développeur, support et opérateur.

INTERFACES DANS LE CODE

`GET /executions/{id}?showAllLogs=...` ; `GET /dataflows/{dataflowDefinitionId}/transformations/{transformationId}/logs` ; `DataflowJobExecutionLogRepository` ; `LogUtils`.

DONNÉES ET ÉTATS

Chunks PostgreSQL puis logs finaux dans les détails du job ; IDs `run/job/step` et `createdAt`.

DÉPENDANCES ET ÉCHANGES

Messages Python, résultats HTTP du runtime, repository des jobs et fonctions de filtrage.

COMPOTEMENT CIBLE PROPOSÉ

Audit/Runtimes Rust conserve des logs structurés bornés avec tenant, run, tentative, étape et offsets. Les gros volumes vont en objet immuable ; PostgreSQL garde index et reçus. Le filtrage de secrets précède diffusion et stockage des vues utilisateur.

MIGRATION DE CETTE CAPACITÉ

Préserver formats et choix de filtre requis par l'interface, documenter explicitement la troncature et migrer les historiques accessibles. Ne pas assimiler l'agrégation actuelle de lignes à une piste d'audit de publication.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Produire un log volumineux, un message final retardé et une valeur secrète de test. Le log reste consultable par pages, signale sa borne, conserve le final une seule fois et ne diffuse pas le secret.

Références de l'analyse

`monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/execution/repository/dao/DataflowJobExecutionLogDAO.kt:9` Stockage des chunks par run/job/étape.

`monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/ExecutionUpdateStatusCommandHandler.kt:170` Finalisation puis nettoyage des chunks.

`monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/ExecutionUpdateStatusCommandHandler.kt:217` Messages retardés et borne des logs.

`monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/util/LogUtils.kt:5` Normalisation, filtre et borne de lignes.

`monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SubmittedCommandHandler.kt:63` Extraction des sorties de session.

PD-09 **Cible non qualifiée****Annulation et signal de commit Spark****COMPOTEMENT OBSERVÉ**

DELETE /executions/{id} est idempotent pour une exécution déjà finale. Pour un job unique actuellement marqué Spark committed, l'annulation est refusée. Pour plusieurs jobs avec un job committé et des travaux en attente, le service annule mais remonte une exception de commit partiel. Le marqueur est posé par un callback authentifié qui charge la dernière exécution de la version, vérifie le projet persistant et le job actif. L'annulation inscrit ExecutionCancelled, passe le run en CANCELED et émet l'événement. Une boucle retire ensuite les files et tente l'annulation distante. Ce booléen ne constitue pas une preuve d'audit ni un historique de publications approuvées.

UTILISATEURS ET RÔLES

Utilisateur qui annule, runtime appelant le callback et IAM.

INTERFACES DANS LE CODE

DELETE /executions/{id} ; POST /spark-hook/versions/{versionId}/scripts/{scriptId}/commit avec contrat pylib ; DataflowExecutionCancelCommandHandler ; SparkCommitCommandHandler.

DONNÉES ET ÉTATS

Flag isSparkCommitted sur job, run CANCELED, commande ExecutionCancelled et files supprimées ; résultat partiel possible.

DÉPENDANCES ET ÉCHANGES

IAM runner authorize, PostgreSQL, runtime HTTP et événements d'exécution.

COMPORTEMENT CIBLE PROPOSÉ

Runtimes Rust définit l'annulation par phase. Avant publication, elle arrête la tentative et abandonne le candidat. Après publication, elle n'efface pas l'édition approuvée. Publisher conserve le reçu exact ; une compensation devient une nouvelle commande audité.

MIGRATION DE CETTE CAPACITÉ

Importer isSparkCommitted comme signal historique limité. Remplacer la résolution dernière-exécution-par-version par run/tentative explicites. Garder un reçu d'annulation distante jusqu'à confirmation au lieu de déduire le succès de la suppression de file.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Annuler avant audit, entre audit et publication, puis après publication. Les deux premiers cas ne changent pas l'édition visible ; le dernier conserve l'édition et retourne son reçu. Un callback d'une vieille tentative ne peut marquer un nouveau run.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowExecutionCancelCommandHandler.kt:115](#)
Idempotence et règles committed/partial committed.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SparkCommitCommandHandler.kt:30](#) Autorisation, recherche et flag commit.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/CancelExecutionCommandHandler.kt:43](#) Retrait des files et annulation distante.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/execution/controller/SparkHookController.kt:23](#) Callback transactionnel du runner.

PD-10 **Cible non qualifiée****Planification cron, fuseaux et récupération des horaires****COMPORTEMENT OBSERVÉ**

Le service conserve les schedules et exige une version en production non supprimée pour les modifier. La commande temporelle accepte notamment MANUAL et RECURRENT ; elle refuse PLANNED et les types événementiels dans ce parcours. Les schedules temporels existants sont diffés tandis que les schedules événementiels sont préservés. Quartz conserve jobs et triggers dans PostgreSQL, demande recovery et interdit une exécution concurrente d'un même job. L'adaptateur normalise certaines expressions, applique le fuseau et peut relancer les schedules dont une prochaine occurrence existe au démarrage. Le contrôle initial de cron utilise directement CronExpression.isValidExpression, distinct du chemin de normalisation. Un listener journalise les misfires ; il ne décrit pas à lui seul une politique métier exhaustive de rattrapage.

UTILISATEURS ET RÔLES

Utilisateur planifiant un flow et opérateur du scheduler.

INTERFACES DANS LE CODE

POST/GET /dataflows/{id}/schedules ; DataflowScheduleTimeCommand ; DataflowSchedulerServiceQuartzImpl ; DataflowJob.

DONNÉES ET ÉTATS

DataflowSchedule avec fréquence, expression, timezone et date ; tables Quartz JDBC avec état de cluster.

DÉPENDANCES ET ÉCHANGES

Version de production, PostgreSQL/Quartz, contrôleur de runs et événements de schedules vers data-dev.

COMPORTEMENT CIBLE PROPOSÉ

Runtimes Rust possède un scheduler durable. Chaque occurrence est identifiée par schedule/version/instant logique, avec politique explicite de DST, retard, chevauchement, rattrapage et annulation. Le déclenchement produit une commande idempotente.

MIGRATION DE CETTE CAPACITÉ

Exporter les expressions et fuseaux puis comparer une séquence d'occurrences avant bascule. Décider le support du type PLANNED historique et des formats acceptés. Éviter la double activation Quartz/Rust pour un même schedule.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Comparer les occurrences sur changement d'heure, arrêt prolongé et redémarrage de deux contrôleurs. Les occurrences exécutées, ignorées ou rattrapées ont chacune un reçu et aucune occurrence n'est créée deux fois.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/schedule/command/DataflowScheduleCommandHandler.kt:104](#) Diff des schedules et types refusés.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/schedule/service/quartz/DataflowSchedulerServiceQuartzImpl.kt:87](#) Création Quartz avec recovery.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/schedule/service/quartz/DataflowSchedulerServiceQuartzImpl.kt:175](#) Validation, normalisation, fuseau et reprise.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/schedule/service/quartz/DataflowSchedulerServiceQuartzImpl.kt:217](#) Job non concurrent transactionnel.

[monorepo · services/process-data/app/data-exec-app/src/main/resources/application.properties:292](#) Quartz JDBC clusterisé.

PD-11 Cible non qualifiée**Déclenchements par fin de flow, dataset ou fichier****COMPORTEMENT OBSERVÉ**

Le consumer d'exécution traite FINISHED en recherchant les schedules dépendants. Seules les versions de branche par défaut fournissent des candidats. Selon le type de script, les candidats identifient le flow et éventuellement un dataset ; les opérations techniques hors collecte sont exclues. Pour chaque schedule, le handler cherche la dernière version en production et déclenche si elle n'est pas supprimée, si un cluster est disponible et si sa dernière exécution n'est pas active. Un endpoint projet/fichier cherche les schedules par projet et chemin. Le déclenchement planifié temporel exige dans le chemin lu une précédente exécution de production. Le handler de fin lu ne filtre pas lui-même le succès de l'exécution source ; la sémantique FINISHED doit être prise en compte.

UTILISATEURS ET RÔLES

Producteur d'événement de fichier, runtime amont et scheduler.

INTERFACES DANS LE CODE

POST /dataflows/events/projects/{id} ; EventExecutionFinishedCommand ;
DataflowVersionRunProjectIdCommand ; TriggerAtEventEvent.

DONNÉES ET ÉTATS

DataflowSchedule avec eventTriggerId/path ; nouveau run avec triggerId, triggerDisplayName et principal SCHEDULED_OWNER.

DÉPENDANCES ET ÉCHANGES

Événements process-data et data-dev, données de projet, versions de production et clusters.

COMPORTEMENT CIBLE PROPOSÉ

Flows/Runtimes Rust distinguent événement technique de fin, publication approuvée et événement source. Les dépendances de données se basent sur une édition Iceberg approuvée. Un flux éphémère est d'abord écrit dans un journal durable ; son epoch/range déclenche des runs bornés sous WAP.

MIGRATION DE CETTE CAPACITÉ

Inventorier quels flows dépendent actuellement d'une fin même échouée et proposer une règle explicite. Donner un ID durable aux événements de fichiers et une clé de déduplication par occurrence. Ne pas perdre les triggers consommés pendant une période de désactivation.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un calcul amont qui échoue n'active pas une dépendance déclarée publication réussie. Rejouer la même édition ou le même fichier ne crée pas un doublon. Une nouvelle occurrence légitime du même chemin crée un nouveau run.

Références de l'analyse

[monorepo](#) · [services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowVersionRunCommandHandler.kt:231](#)

Précondition de dernière exécution pour cron.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowVersionRunCommandHandler.kt:288](#)

Déclenchement sur fin et projet/fichier.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/schedule/aggregate/ScheduleCandidate.kt:14](#) Candidats selon type de script.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/schedule/controller/ScheduleController.kt:60](#) Route de fichier projet.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/event/consumer/ProcessDataKafkaEventConsumer.kt:38](#) Traitement de FINISHED.

PD-12 **Cible non qualifiée****Profil de clusters, capacité et commandes de cycle de vie****COMPORTEMENT OBSERVÉ**

Les API créent, vérifient, mettent à jour, listent, arrêtent, reprennent et suppriment des clusters. Deux familles de configuration existent : Python avec CPU/RAM/image, et PySpark avec ressources du driver et des exécuteurs. La validation vérifie notamment nom disponible, image compatible, taille mémoire minimale et au moins une session. Une mise à jour ne peut changer de famille ; un changement de configuration remplace les sessions, tandis qu'un changement de nombre augmente ou réduit leur effectif. La réduction préfère les sessions supprimables puis marque les autres à supprimer. Les clusters par défaut sont protégés de certaines

commandes de pause et suppression. La pause mémorise le nombre souhaité de sessions pour la reprise. L'état du cluster est dérivé des états des sessions. Le parcours `/projects/{projectId}/execution-clusters` restitue les profils disponibles sans appliquer de filtre projet dans le handler de liste lu.

UTILISATEURS ET RÔLES

Administrateur des ressources, utilisateur choisissant un profil et moteur de planification.

INTERFACES DANS LE CODE

POST `/clusters/check` ; GET/POST `/clusters` ; GET/PATCH/DELETE `/clusters/{clusterId}` ; POST `/clusters/{clusterId}/stop` et `/start` ; GET `/clusters/{clusterId}/sessions` ; GET `/projects/{projectId}/execution-clusters` ; GET `/projects/{projectId}/execution-clusters/{clusterId}/libraries`.

DONNÉES ET ÉTATS

Cluster avec owner, configuration, default, paused, toDelete, nombre avant pause et image de serveur ; collection de Session avec état. La suppression physique du cluster attend la disparition de ses sessions.

DÉPENDANCES ET ÉCHANGES

Catalogue d'images et bibliothèques, app-management pour les ressources, événements de cluster vers data-dev et historique d'exécutions.

COMPORTEMENT CIBLE PROPOSÉ

Runtimes Rust remplace les clusters persistants par des profils de capacité et des pools de workers fongibles. Les quotas organisation/projet, admission, priorité et limites par run sont explicites. Python utilisateur reçoit un profil isolé ; aucun profil final ne dépend de Spark.

MIGRATION DE CETTE CAPACITÉ

Conserver des identifiants et noms de profil pour les flows importés, traduire les ressources utiles et signaler les options Spark sans équivalent. Distinguer arrêter l'admission, drainer les tentatives et détruire la capacité. Décider si les profils sont globaux ou limités au projet.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Réduire un pool pendant deux runs : les runs admis terminent ou sont annulés selon la politique et aucune session occupée n'est perdue implicitement. Un projet à quota épuisé reste en attente avec raison visible, puis démarre après libération. Le changement de moteur exige requalification du manifeste.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/session/controller/ClusterController.kt:80](#) Toutes les commandes de cluster et lectures de sessions.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/session/aggregate/Cluster.kt:40](#) Remplacement, réduction, pause et reprise des sessions.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/session/aggregate/Cluster.kt:208](#) Configurations et validations de ressources.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/session/command/UpdateClusterCommandHandler.kt:36](#) Famille du cluster immuable.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/session/controller/ExecutionClusterController.kt:27](#) Route de sélection de profil dans un projet.

Réconciliation des sessions et des serveurs Spark Connect

COMPORTEMENT OBSERVÉ

L'orchestrateur compare les sessions persistées, les files d'exécution et les états remontés par app-management. Pour PySpark, un serveur Spark Connect est assuré tant que le cluster conserve des sessions. Une nouvelle image provoque le remplacement du serveur. La réconciliation récupère un inventaire des sessions et serveurs, actualise les images effectivement déployées et oublie un serveur absent pour le recréer. Les sessions mortes ou revenues à STARTING/NOT_STARTED font échouer leurs jobs, retirent leurs soumissions et peuvent conduire à une annulation distante. Le log de ce chemin emploie un message out-of-memory même si la cause exacte n'est pas établie ici. L'adaptateur suit aussi localement le dernier retour des sessions et applique des délais distincts de démarrage et de silence. Ces caches de temps et l'état désiré PostgreSQL n'ont pas le même périmètre de reprise.

UTILISATEURS ET RÔLES

Orchestrateur process-data et opérateur d'exploitation.

INTERFACES DANS LE CODE

SessionCheckCommandHandler ; SessionGroupingResultBuilder ; REST app-management
/private/sessions/reconcile, /private/sessions/start, /private/sessions/delete, /private/sessions/server/start et /private/sessions/server/delete.

DONNÉES ET ÉTATS

Session avec sessionKey, startedAt, updatedAt, deployedImage et flags de suppression ; Cluster.serverImage ; files des jobs ; caches locaux de dernier retour dans l'adaptateur.

DÉPENDANCES ET ÉCHANGES

App-management et runtime Python/PySpark ; catalogue d'images ; journal d'exécutions et événements de sessions.

COMPORTEMENT CIBLE PROPOSÉ

Runtimes Rust réconcilie tentative, bail et worker observé. Un worker disparu devient une tentative perdue, avec cause structurée et nouvelle tentative décidée par politique. L'ancien worker ne peut plus publier : Writer/Publisher vérifient l'autorité de tentative avant toute transition approuvée.

MIGRATION DE CETTE CAPACITÉ

Importer les diagnostics et règles utiles de nettoyage ; supprimer entièrement Spark Connect à la fin de migration. Remplacer les caches de silence significatifs par des heartbeats et échéances persistés. Prévoir un inventaire Kubernetes encapsulé et une réconciliation sans dépendre d'un nom d'image mutable.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Perdre un pod, un serveur et une réponse d'inventaire à des moments différents. La disparition est distinguée d'une indisponibilité de l'API. Une reprise ne crée pas deux publications ; les octets d'une tentative expirée restent candidats non approuvés et sont traités par rétention.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SessionCheckCommandHandler.kt:107](#) Cycle de serveur Spark Connect selon sessions et image.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SessionCheckCommandHandler.kt:234](#) Échec des jobs attachés aux sessions mortes.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SessionCheckCommandHandler.kt:286](#) Réconciliation de l'inventaire et serveurs absents.

[monorepo](#) · [services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/session/service/appmanagement/ClusterConnectServerServiceAppManagementImpl.kt:85](#) Polling, délais et inventaire distant.

PD-14 Cible non qualifiée

Images d'exécution, builds et historique des mises à jour

COMPORTEMENT OBSERVÉ

Une image personnalisée est créée à partir d'une image de base avec des bibliothèques validées. L'appel de build est délégué à app-management ; l'état et l'historique de mise à jour sont conservés localement. Une image de base ne suit pas le parcours de modification/suppression des images personnalisées. Une mise à jour refuse une construction encore RUNNING et refuse une liste inchangée. Le rebuild repart du dernier besoin exprimé, y compris après un échec, et peut retargeter le registre interne configuré. Les callbacks de build ne modifient l'image que si le dernier update est RUNNING et correspond exactement au nom/tag attendu. Au démarrage, les images de base sont synchronisées ; les changements de tags peuvent redémarrer les sessions. Les images personnalisées construites avant une base actualisée peuvent être reconstruites si aucun build ne tourne déjà. La suppression d'image déclenche aussi la suppression des clusters concernés.

UTILISATEURS ET RÔLES

Administrateur et auteur d'un environnement Python personnalisé.

INTERFACES DANS LE CODE

POST/GET /images ; PATCH/DELETE/GET /images/{imageId} ; POST /images/{imageId}/builds ; GET /images/{imageId}/updates ; ImageServiceAppManagement ; UpdateImageStatusCommandHandler.

DONNÉES ET ÉTATS

Image, ImageUpdate et historique incrémental de bibliothèques ; RUNNING puis SUCCESS/FAILURE ; nom de base, nom d'image construite et dates de build.

DÉPENDANCES ET ÉCHANGES

App-management /private/build/image, registre de conteneurs, callbacks Kafka de build, images de base configurées et clusters.

COMPORTEMENT CIBLE PROPOSÉ

Runtimes/Applications Rust conservent une spécification d'environnement immuable et un digest de résultat, avec provenance, SBOM, politique de sécurité et reçu de build. Les builders tiers peuvent être Go/C++ ou outils sans JVM encapsulés. Le run référence le digest qualifié.

MIGRATION DE CETTE CAPACITÉ

Exporter historique, base et besoins de packages, puis construire des environnements sans Spark/JVM. Ne pas assimiler nom/tag à identité immuable. Traiter les échecs et callbacks retardés comme des tentatives de build distinctes. Conserver les images legacy uniquement pendant la transition contrôlée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un callback ancien reçu pendant une reconstruction n'écrase pas le nouveau résultat. Une base changée crée un nouveau digest et nécessite la requalification attendue ; les runs déjà admis conservent leur digest d'origine. La suppression d'une image encore retenue par l'audit est refusée ou différée.

Références de l'analyse

[monorepo](#) · [services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/image/controller/ImageController.kt:58](#) Contrats de création, mise à jour, rebuild et lecture.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/image/command/UpdateImageCommandHandler.kt:42](#) Restrictions et diff des besoins.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/image/command/RebuildImageCommandHandler.kt:44](#) Reconstruction après échec et reprise des besoins.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/image/command/UpdateImageStatusCommandHandler.kt:130](#) Protection contre callback obsolète.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/image/command/RebuildOutdatedCustomImagesCommandHandler.kt:49](#) Reconstruction des images dépendantes devenues anciennes.

PD-15 **Cible non qualifiée**

Validation et consultation des bibliothèques Python

COMPOTEMENT OBSERVÉ

La validation compare les bibliothèques demandées avec celles de l'image de base. Une bibliothèque de base imposée ne peut être redéfinie avec une version contradictoire. Les bibliothèques personnalisées sont résolues puis enregistrées avec leur contrainte demandée. Les erreurs sont agrégées pour retourner plusieurs problèmes à la fois ; une demande sans bibliothèque personnalisée est refusée dans ce service de vérification. L'adaptateur consulte les métadonnées PyPI ou un index particulier pour la bibliothèque interne. Il normalise les noms/extras et vérifie les versions et extras quand les métadonnées permettent de conclure. Une absence de métadonnées extras n'équivaut pas à une validation complète. Les lectures distinguent bibliothèques appliquées à l'image et besoins du dernier update.

UTILISATEURS ET RÔLES

Auteur d'environnement et administrateur du registre.

INTERFACES DANS LE CODE

GET /images/{imageId}/libraries?lastUpdate=... ; GET /projects/{projectId}/execution-clusters/{clusterId}/libraries ; LibraryService.checkLibraries ; LibraryServicePyPiImpl.

DONNÉES ET ÉTATS

Library et relations base/custom/update avec nom, contrainte et version résolue ; historique des demandes d'environnement.

DÉPENDANCES ET ÉCHANGES

Index Python public/interne, image de base, service de build et réseau sortant du service.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes Rust définit le lock d'environnement, tandis qu'un builder Python encapsulé résout les dépendances. Le lock fixe versions transitives, hashes, provenance et plateforme. Egress gère l'accès aux index ; Audit conserve le besoin demandé et le résultat exact.

MIGRATION DE CETTE CAPACITÉ

Rejouer les demandes historiques contre un index contrôlé, détecter les packages disparus, natifs ou dépendants de Java, puis qualifier les locks. Préserver les contraintes initiales comme information sans les utiliser seules pour reproduire un run.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une contrainte inchangée produit le même environnement quand le lock existe, même si l'index publie une nouvelle version. Un package lié à la JVM ou un extra inconnu est bloqué avant activation selon la règle de qualification ; une indisponibilité d'index reste une erreur distincte d'un package inexistant.

Références de l'analyse

[monorepo](#) · [services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/image/service/LibraryService.kt:20](#) Contrôle des packages de base, custom et agrégation des erreurs.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/image/service/LibraryServicePyPiImpl.kt:28](#) Résolution des métadonnées et index.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/image/service/LibraryServicePyPiImpl.kt:89](#) Normalisation et traitement des extras.

[monorepo](#) · [services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/image/controller/ImageController.kt:134](#) Lecture appliquée ou dernier update.

PD-16 **Cible non qualifiée****Historique des exécutions, filtres, facettes et consultation****COMPOTEMENT OBSERVÉ**

POST /executions exige le projet, convertit les identifiants IAM/legacy et retourne une recherche paginée d'exécutions visibles. Les critères entre dimensions sont combinés par AND et les valeurs d'une dimension par OR. Les facettes ignorent leur propre sélection pour présenter les alternatives. La recherche peut viser noms d'exécution/flow, branches, clusters, utilisateurs, statuts et bornes temporelles. Le repository filtre les versions techniques selon toShow et utilise la date de création pour les bornes. Les tris autorisés sont contrôlés dans le controller. Les autres routes servent l'historique d'une branche, le détail d'un run, les logs d'une transformation, les exécutions d'un cluster et les exécutions RUNNING. Le paramètre showAllLogs influe sur la restitution des logs, pas sur leur autorisation.

UTILISATEURS ET RÔLES

Développeur, opérateur et frontends.

INTERFACES DANS LE CODE

POST /executions ; GET /executions/{executionId} ; GET /dataflows/{dataflowId}/branches/{branchName}/executions ; GET /dataflows/{dataflowDefinitionId}/transformations/{transformationId}/logs ; GET /clusters/{clusterId}/executions ; GET /clusters/executions/running.

DONNÉES ET ÉTATS

DataflowExecution, JobExecution, versions toShow, utilisateurs/projets miroirs et logs stockés ; DTO détail et réponses paginées/facettes.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, canonicalisation IAM, agrégats d'exécution et authorization gateway.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes/Audit Rust exposent une vue indexée des runs, tentatives et transitions, liée aux éditions approuvées sans confondre statut technique et publication. Les totaux, facettes et logs appliquent la même portée d'autorisation que les lignes.

MIGRATION DE CETTE CAPACITÉ

Reproduire un corpus de requêtes UI et comparer tri, pagination, filtres vides, fenêtres temporelles et versions cachées. Ajouter les IDs de tentative et de publication de manière additive avant de retirer les DTO Spark/session.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Comparer historique et facettes sur des runs de plusieurs projets, branches, statuts et utilisateurs. Les données invisibles ne changent ni les options ni les compteurs. Un run techniquement terminé sans publication reste distinguable d'une édition approuvée.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/execution/controller/DataflowPipelineController.kt:90](#) Recherche, facettes, tri et canonicalisation.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/execution/controller/DataflowPipelineController.kt:147](#) Historique, détail et logs.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/execution/repository/DataflowExecutionFilteredQuery.kt:35](#) Prédicats, dates, recherche et toShow.

[monorepo · services/process-data/app/data-exec-](#)

[domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/query/ExecutionFilter.kt:6](#) Dimensions du filtre.

PD-17 **Cible non qualifiée****Suivi temps réel SSE et notifications de clusters****COMPOTEMENT OBSERVÉ**

Les routes exposent un abonnement SSE par utilisateur, version de flow et éventuellement exécution, ainsi que des routes pour un cluster ou toutes les exécutions en cours. Les emitters sont conservés dans une map du processus et réutilisés pour une même clé. Ils expirent et sont supprimés sur erreur, fin ou timeout. Le chemin d'émission lu envoie directement aux clés de flow compatibles ; les données de cluster et de toutes les exécutions sont relayées vers Notification Center par Kafka. Les clés SSE de cluster peuvent être enregistrées, mais ce chemin n'émet pas directement vers elles. L'état SQL et les endpoints de détail restent nécessaires pour rétablir l'affichage après reconnexion ; aucun replay durable par Last-Event-ID n'a été constaté dans ce service.

UTILISATEURS ET RÔLES

Frontend de développement et console d'exploitation.

INTERFACES DANS LE CODE

GET /dataflows/{dataflowId}/versions/{versionId}/sse ; GET /clusters/{clusterId}/executions/sse ; GET /clusters/executions/running/sse ; SseEmitterService ; NotificationCenterRelayKafkaImpl.

DONNÉES ET ÉTATS

Emitters locaux indexés par SseKey ; DTO d'état construit au fil des événements ; relay Kafka vers rooms de cluster.

DÉPENDANCES ET ÉCHANGES

Runtimes d'exécution, serveur HTTP/SSE, Notification Center et Kafka.

COMPOTEMENT CIBLE PROPOSÉ

Notifications Rust fournit un flux de mises à jour autorisé avec curseur durable et reprise. Les messages signalent un changement identifié ; les détails sont relus avec contrôle de portée. Les clients gèrent doublons, réordonnancement et perte de connexion.

MIGRATION DE CETTE CAPACITÉ

Confirmer les routes effectivement utilisées par les interfaces et décider le retrait des anciens abonnements cluster SSE. Introduire une séquence de changement par run et une resynchronisation explicite. Retirer Kafka JVM lorsque le nouveau journal/relay sans JVM est qualifié.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Déconnecter le client pendant plusieurs transitions puis reconnecter sur un autre pod. Il récupère toutes les transitions utiles ou un signal explicite de resynchronisation, sans accéder à un run dont le droit a été retiré. Deux onglets d'un même utilisateur ne se neutralisent pas.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/execution/controller/DataflowPipelineController.kt:248](#) Abonnement par flow/version et exécution optionnelle.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/session/controller/ClusterController.kt:185](#) Routes SSE de clusters et exécutions actives.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/execution/service/SseEmitterService.kt:39](#) Map locale et cycle de vie des emitters.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/execution/service/SseEmitterService.kt:94](#) Émission flow directe et relais cluster vers Notification Center.

PD-18 **Cible non qualifiée****Autorisation des runners et configuration lakehouse****COMPORTEMENT OBSERVÉ**

Deux opérations techniques sont contrôlées par RunnerAuthorizationClient : la lecture de configuration et le signal de commit. Le client exige une identité BEARER_JWT et un clientId ; il appelle IAM avec les credentials entrants, l'utilisateur, le client, l'opération et éventuellement le projet. Une réponse négative donne un refus ; un IAM indisponible donne une erreur de service. Le client n'a pas de cache ou d'autorisation de secours dans le code lu. /lakehouse/conf retourne les noms de catalogue et namespace après autorisation et exige le User-Agent pylib. Le hook commit reprend le projet du run trouvé puis vérifie l'opération dédiée. Le User-Agent est un critère de route, pas une preuve d'identité autonome.

UTILISATEURS ET RÔLES

Runner Python/PySpark et IAM.

INTERFACES DANS LE CODE

GET /lakehouse/conf ; POST /spark-hook/versions/{versionId}/scripts/{scriptId}/commit ; RunnerAuthorizationClient.requireConfiguration ; RunnerAuthorizationClient.requireCommit ; IAM /internal/runners/authorize.

DONNÉES ET ÉTATS

Décision centrale à la demande ; contexte d'identité entrante ; signal isSparkCommitted sur le job. Aucune preuve de publication Iceberg n'est créée par la réponse de configuration.

DÉPENDANCES ET ÉCHANGES

IAM runner authorization, bearer entrant et paramètres de catalogue configurés.

COMPOTEMENT CIBLE PROPOSÉ

IAM/Runtimes Rust délivrent des capacités de run limitées au projet, à la tentative, aux entrées, sorties et à l'échéance. Query contrôle la lecture ; Writer médie le staging ; Publisher seul approuve la publication. Le worker n'obtient jamais table.modify générique qui permet aussi commit/drop.

MIGRATION DE CETTE CAPACITÉ

Remplacer le hook Spark par un protocole explicite de proposition de candidat et de publication. Propager runId/tentativeId et portée réelle jusque dans les appels de données. Conserver une façade de configuration temporaire avec les mêmes refus explicites.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une identité de service sans délégation de run, un mauvais projet, un client non prévu ou une capacité expirée ne peuvent ni écrire ni publier. Une panne IAM échoue sans fallback permissif. Le worker ne peut pas utiliser son accès staging pour supprimer la table.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/configuration/RunnerAuthorizationClient.kt:26](#) Opérations et appel central, refus et indisponibilité.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/configuration/RunnerAuthorizationClient.kt:53](#) Exigence bearer JWT et clientId.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/execution/controller/ConfigurationForLibController.kt:19](#) Configuration limitée après autorisation.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/SparkCommitCommandHandler.kt:30](#) Projet persisté utilisé pour la décision commit.

PD-19 **Cible non qualifiée****Miroirs IAM, identifiants canoniques et réconciliation des droits****COMPOTEMENT OBSERVÉ**

Le service peut consommer l'identité legacy ou IAM selon une propriété exclusive. Les événements IAM créent/actualisent les utilisateurs et projets, lient leurs identifiants et alimentent les adhésions. Une suppression utilisateur désactive le miroir. Une suppression projet laisse le miroir en place car versions et exécutions le référencent. Le transfert de propriétaire arrive via les événements de data-dev, pas directement via l'offboarding IAM. Une adhésion dont les correspondances sont indisponibles est refusée et suit retry/DLT. Les API résolvent les projets IAM vers les IDs historiques pour interroger les tables, puis réexposent l'ID canonique si connu. Une route d'administration lance une réconciliation asynchrone des faits d'autorisation.

UTILISATEURS ET RÔLES

IAM, gateway et administrateur de migration.

INTERFACES DANS LE CODE

IamKafkaEventConsumer ; ProjectIdResolver ; POST /admin/reconcile-tuples.

DONNÉES ET ÉTATS

User/Project/ProjectMember miroirs, tables de correspondance IAM/legacy et faits d'autorisation émis.

DÉPENDANCES ET ÉCHANGES

IAM, événements data-dev de propriétaire, PostgreSQL et Kafka.

COMPOTEMENT CIBLE PROPOSÉ

IAM Rust reste l'autorité ; Runtimes garde uniquement les références canoniques et l'attribution historique. Les contrôles de run utilisent la décision courante et une capacité bornée.

MIGRATION DE CETTE CAPACITÉ

Importer les correspondances, réparer les événements refusés puis basculer une seule source d'identité. Préserver les auteurs historiques quand une identité est supprimée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un membre reçu avant son projet est rejoué après réconciliation sans création de faux projet. Une suppression utilisateur interdit les nouveaux runs et conserve les historiques attribués.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/event/consumer/IamKafkaEventConsumer.kt:56](#) Miroirs, suppression et membres.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/projectid/ProjectIdResolver.kt:58](#) Conversion des identifiants entrants et sortants.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/admin/controller/AdminReconcileTuplesController.kt:49](#) Réconciliation administrative asynchrone.

PD-20 **Cible non qualifiée**

Outbox, déduplication des événements et quarantaine**COMPOTEMENT OBSERVÉ**

Les producteurs utilisent une outbox transactionnelle. Le relay réclame les lignes non envoyées avec SKIP LOCKED, attend l'acquittement Kafka puis marque sentAt ; un échec conserve la ligne, incrémente attempts et stocke l'erreur. Le tri par date ne constitue pas la barrière par clé présente dans data-dev. Des consumers emploient ProcessEventInbox dans la transaction : la clé est le messageId ou une empreinte de payload, unique par type d'événement. La couverture n'est pas universelle à tous les listeners. Les dead letters sont conservées avec provenance, payload et erreur ; un échec de stockage est relancé pour empêcher l'acquittement. La rétention supprime seulement les dead letters résolues et les lignes outbox envoyées selon des délais distincts.

UTILISATEURS ET RÔLES

Consumers interservices et exploitation.

INTERFACES DANS LE CODE

OutboxRelay.drain ; ProcessEventInbox.claim ; DeadLetterEventConsumer.consume ; DeadLetterRetention.cleanupResolved.

DONNÉES ET ÉTATS

Outbox SQL, inbox SQL, dead_letter_event et offsets Kafka ; livraison au moins une fois et déduplication locale pour les handlers équipés.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Kafka, événements data-dev/process-data/app-management/IAM et métriques.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes/Audit Rust utilisent un journal durable sans JVM, une inbox et des séquences par agrégat. Les effets externes ont leur reçu idempotent. L'acquittement d'une progression de données suit la publication approuvée.

MIGRATION DE CETTE CAPACITÉ

Cartographier chaque listener et sa clé d'idempotence, importer les éléments non acquittés, puis arrêter l'ancien producteur avant de transférer une partition logique. Définir explicitement l'ordre par run.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Couper après acquittement transport mais avant commit SQL du relay : le doublon est accepté sans double effet. Rejouer une dead letter après réparation conserve son lien avec l'événement initial et son résultat.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/outbox/OutboxRelay.kt:25](#) Envoi acquitté, tentatives et rétention.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/outbox/OutboxEventJPA.kt:18](#) Réclamation ordonnée avec SKIP LOCKED.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/event/consumer/ProcessEventInbox.kt:14](#) Déduplication dans la transaction.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/deadletter/DeadLetterEventConsumer.kt:37](#) Quarantaine persistante et refus d'ack si échec.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/common/deadletter/DeadLetterRetention.kt:22](#) Nettoyage limité aux éléments résolus.

PD-21 **Cible non qualifiée****Démarrage, reprise des files et coexistence des réplicas****COMPORTEMENT OBSERVÉ**

Au démarrage de production, le service cherche un autre pair vivant dans les tables Quartz. Si un pair existe, il évite les opérations de cold start qui annuleraient les runs et réinitialiseraient clusters/schedules. Sinon, il annule les runs orphelins tout en conservant ceux dont une file durable prouve une progression reprenable, initialise les clusters si activés et relance les schedules selon les flags. La disponibilité app-management est attendue avec un nombre borné de tentatives. Les images et bibliothèques sont synchronisées même en présence d'un pair. La détection de pair et les protections SQL observées ne qualifient pas à elles seules toute la boucle d'orchestration pour plusieurs réplicas.

UTILISATEURS ET RÔLES

Opérateur du déploiement et orchestrateur.

INTERFACES DANS LE CODE

`RunAfterStartup.resetSessions` ; `StartCluster.startClusters` ; `ClusterPeerCheck.hasOtherLivePeer` ; `DataflowExecutionCancelRunningCommand`.

DONNÉES ET ÉTATS

Files persistées, exécutions actives, état Quartz/heartbeat et images ; état local de la boucle d'orchestration.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Quartz, app-management, Kafka et readiness des runtimes.

COMPORTEMENT CIBLE PROPOSÉ

Runtimes Rust utilise des baux persistants et une réconciliation idempotente par run/tentative. Un rolling update ne change pas la propriété d'une publication ni l'identité de sa tentative. Les workers ne possèdent aucun état métier durable indispensable.

MIGRATION DE CETTE CAPACITÉ

Qualifier crash et rolling update pour chaque étape de file avant de migrer les runs actifs. Importer les intentions restantes ou laisser finir les runs legacy ; empêcher l'activation simultanée de deux orchestrateurs sur un même run.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Redémarrer tous les contrôleurs puis effectuer un rolling update avec des jobs en attente, soumis et terminés mais non finalisés. Chaque run retrouve une issue expliquée et au plus une publication approuvée.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/RunAfterStartup.kt:94](#) Différence cold start/rolling update et flags.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/RunAfterStartup.kt:54](#) Attente bornée d'app-management.

[monorepo · services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/common/cluster/ClusterPeerCheck.kt:24](#) Détection de pair Quartz.

[monorepo · services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/command/DataflowExecutionCancelCommandHandler.kt:179](#) Conservation des runs possédant une progression durable.

PD-22 **Cible non qualifiée****Maintenance SQL, migrations et exploitation du service****COMPOTEMENT OBSERVÉ**

Le service comporte les migrations Flyway et un runner de migrations de code. Ce runner marque le résultat dans la table de suivi ; le chemin d'échec demande une sortie Spring avec code 0, ce qui nécessite une vérification explicite côté déploiement. La maintenance planifie VACUUM ANALYZE des tables d'exécution/jobs et collecte des statistiques de taille PostgreSQL. Le chart configure utilisateur non-root, probes et ressources ; le stockage persistant optionnel impose des contraintes de réplica et stratégie de mise à jour documentées. Les caches, outbox, quarantaine, tables Quartz et artefacts S3 ne se sauvegardent pas par un seul mécanisme.

UTILISATEURS ET RÔLES

Exploitation, équipe de migration et administrateur PostgreSQL.

INTERFACES DANS LE CODE

DatabaseMaintenanceService ; CleyropMigrationRunner ; Flyway ; probes et configuration Helm.

DONNÉES ET ÉTATS

Schémas SQL, table de migrations, statistiques, logs et volumes optionnels.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Kubernetes/Helm, S3, métriques et plateforme de sauvegarde.

COMPOTEMENT CIBLE PROPOSÉ

Migration/Maintenance Rust gèrent les opérations suivies, idempotentes et à échec explicite. Les migrations suivent Expand/Migrate/Contract. Audit établit les plans de rétention des candidats et preuves ; compaction de données sous WAP et GC séparé par garde de rétention.

MIGRATION DE CETTE CAPACITÉ

Conserver une sauvegarde restaurable des commandes, historiques et reçus avant conversion. Remplacer le runner de migration JVM et ses codes de sortie ambigus. Documenter la restauration coordonnée avec les éditions Iceberg et artefacts immuables.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une migration échouée bloque l'activation et expose son erreur. Restaurer la base de contrôle avec le catalogue plus récent reconstruit les projections sans faire reculer une édition approuvée ni republier un effet déjà reçu.

Références de l'analyse

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/exec/app/maintenance/DatabaseMaintenanceService.kt:39](#) Maintenance SQL et collecte de statistiques.

[monorepo · services/process-data/app/data-exec-](#)

[app/src/main/kotlin/com/cleyrop/data/codemigration/runner/CleyropMigrationRunner.kt:30](#) Suivi et sortie sur échec de migration.

[monorepo · services/process-data/chart/templates/deployment.yaml:50](#) Exécution non-root et probes du chart.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- API Rust de commandes de run, admission, tentative, annulation, planification et progression, avec idempotence, révision attendue et journal durable.
- Flows fournit une révision immuable et un manifeste typé. Runtimes épingle entrées approuvées, environnement, paramètres, portée IAM, budget et échéance avant admission.
- PostgreSQL reste l'autorité des commandes, identités et reçus d'effets ; les checkpoints publiés se lisent dans les éditions Iceberg approuvées.
- Publisher coordonne les sorties et crée une release multi-sorties elle-même édition Iceberg, comprenant source/epoch/range/participants ; les lecteurs épinglent cette release.
- Le statut du run distingue calcul, staging, audit, publication et effets. EXPORT est un effet ; EXPOSURE est un consommateur déclaré.

Plan de données proposé

- Workers fongibles pour runs bornés, moteurs Rust/Arrow/SQL et Python isolé. Suppression finale des sessions Spark, Spark Connect, Quartz JVM, Kafka JVM et de toute autre dépendance Java.
- Le worker soumet les octets au Writer de confiance. Il ne détient pas table.modify générique ; Publisher contrôle la publication après audit de fichiers immuables.
- Tous les kinds TABLE, INCREMENTAL, VIEW, FILESET, MODEL, EMBEDDINGS, FEATURESET et STREAM utilisent WAP. Un flux éphémère est journalisé durablement avant gouvernance et traité par intervalles bornés.
- Le candidat porte origin.* ; la preuve post-candidat est liée durablement sans réécriture du summary par fast-forward. UUID/refs/schema/spec/ascendance ne remplacent pas un protocole de fencing Hemera.
- Publish-then-ack lie checkpoint publié et édition approuvée. Compaction suit le même circuit WAP ; expiration/GC utilisent un plan audité et les gardes de rétention.

Découpage du core Rust

- runtimes-api : DTO compatibles, admission, autorisations et résolution des identités canoniques.

- run-controller : machine d'états, files persistées, baux de tentative, réconciliation, annulation et receipts.
- scheduler : occurrences durables, cron/fuseaux, événements, déduplication et politique de rattrapage.
- environment-registry : locks Python, digests de conteneur, provenance et builders encapsulés.
- worker-gateway : lancement, état, logs structurés et capacités bornées ; aucune API de publication générique vers le worker.
- audit-projection : historique des runs/tentatives, lien aux publications approuvées, métriques et flux de notification reprenables.

Identité, autorisation et isolation

- Conserver la distinction initiateur manuel, owner interne/planifié et projet réel des opérations fabriquées. Ne pas dériver des droits du seul owner courant ou du dataflowId synthétique.
- Les dates de capacité persistées doivent survivre à la reprise. Une nouvelle tentative ne prolonge pas implicitement l'autorisation du run.
- Le code utilisateur, packages, logs, URL signées et fichiers sont des surfaces tenant ; limiter octets, durée, réseau, secrets et scopes, avec redaction dans les diagnostics.
- La suppression de projet/utilisateur conserve l'attribution historique sans conserver un droit d'accès. Les projections et caches ne sont pas une source d'autorisation autonome.

Exploitation et reprise

- La lecture établit les mécanismes présents, pas leur qualification en multi-réplicas. Quartz clusterisé ne suffit pas à prouver la sûreté de la boucle Orchestrator de chaque pod.
- Surveiller âge des files, durée d'admission, tentatives perdues, dead letters, relay attempts, silences runtime, dérive d'image, retard de schedule et différences entre fin technique et publication.
- Sauvegarder PostgreSQL, artefacts et catalogue avec une procédure de restauration qui réconcilie les éditions déjà publiées ; ne pas traiter la base de contrôle comme checkpoint data autoritaire.
- Qualifier redémarrage complet, rolling update, perte réseau après submit, arrêt pendant commit et callback retardé. Aucun de ces parcours n'a été exécuté dans cet audit.

Séquence de migration

- Geler le contrat API/événements et un corpus représentatif : Python, SQL, PySpark, partial, preview, collect, delete, empty et update-column.
- Importer IDs canoniques, versions, manifestes, profils, schedules et historique ; isoler les versions anciennes sans manifeste complet.
- Construire scheduler et controller Rust en observation passive puis comparer leurs décisions sans lancer de travail concurrent.
- Qualifier l'exécution sans JVM par opérateur, package et type de flow ; rendre les incompatibilités explicites avant activation.
- Déployer Writer/Publisher, audit immuable, checkpoints d'édition et receipts ; qualifier les scénarios de panne et de multi-sorties.

- Basculer par projet/profil après drainage ou reprise explicite des runs legacy ; attribuer chaque occurrence et run à un seul controller.
- Réconcilier queues/outbox/DLT puis retirer process-data JVM, Spark Connect, sessions Spark, Quartz et transport Kafka JVM. Conserver les lectures d'historique nécessaires.

Écarts et décisions encore ouverts

- Le contrat exact de succès attendu par les triggers FINISHED doit être décidé ; le handler lu ne filtre pas le succès source.
- Le parcours schedule temporel requiert une exécution antérieure de production : confirmer la création initiale attendue côté produit.
- Le signal isSparkCommitted pointe le dernier run d'une version et un script ; préciser les garanties de corrélation actuelles avant de le remplacer.
- Le submit distant peut réussir avec réponse perdue. La clé d'idempotence réellement garantie par app-management doit être qualifiée.
- La sûreté multi-réplicas de la boucle de files et les limites des verrous observés exigent des tests de panne ; aucune garantie globale exactly-once n'est déduite.
- Les anciennes routes SSE cluster enregistrent des clés qui ne sont pas ciblées par le chemin d'émission direct lu ; vérifier les clients encore concernés.
- Quelles fonctions PySpark, packages natifs et ressources personnalisées n'ont pas d'équivalent sans JVM ? L'inventaire des définitions clientes n'a pas été consulté.
- Les paramètres réellement déployés, volumes, budgets et versions runtime restent inconnus ; le dossier décrit le code du commit de référence.

Références de l'analyse

monorepo · [services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowExecution.kt:55](#) État, identité et lifecycle d'un run.

monorepo · [services/process-data/app/data-exec-domain/src/main/kotlin/com/cleyrop/data/exec/domain/execution/aggregate/DataflowVersion.kt:35](#) Version et manifestes de scripts.

monorepo · [services/process-data/app/data-exec-app/src/main/kotlin/com/cleyrop/data/exec/app/execution/service/Orchestrator.kt:45](#) Boucle opérationnelle.

monorepo · [services/process-data/app/gradle/libs.versions.toml:16](#) Versions Kotlin et Spring déclarées.

monorepo · [services/process-data/app/data-exec-app/build.gradle.kts:25](#) Cible JVM du module applicatif.